

Supplementary Material for “Mask-Aware Certificates for Permutation-Complete Stateless Scan Plans”

Anonymous Authors

Abstract

This supplement fixes the input semantics, expands every proof used by the main paper, specifies the JSON and JSONL contracts, describes the independent oracle and deterministic test families, and gives a clean-room reproduction protocol. It is part of the artifact’s claim boundary. Nothing in this document upgrades plan verification into execution attestation or extends the counting algorithm to arbitrary permutations.

CONTENTS

I	Normative Semantics	2
I-A	Integer and interval conventions	2
I-B	Affine target map	2
I-C	Fragments and phases	2
I-D	Authorization and multiplicity	2
II	Defect Identity: Expanded Proof	3
III	Counting One Fragment	4
III-A	Bounded congruence normalization	4
III-B	Signed floor sum	4
III-C	Residue threshold formula	4
IV	Counting Fragment Pairs	5
V	Prefix Defects and Witnesses	5
VI	Normative JSON Contracts	6
VI-A	Declaration	6
VI-B	Plan	6
VI-C	Certificate	6
VI-D	Canonical encoding and binding	6
VII	Checker Soundness and Completeness	6
VIII	Independent Oracle and Test Design	7
VIII-A	Explicit oracle	7
VIII-B	Deterministic test families	7
IX	Benchmark Construction	8
X	Reproduction Protocol	8
XI	Claim–Evidence Boundaries	8

XII	Known Limitations	9
XIII	Reviewer Checklist	9
XIV	Conclusion	9

I. NORMATIVE SEMANTICS

A. Integer and interval conventions

All mathematical integers are unbounded. The interchange schema restricts the universe size to $1 \leq N \leq 2^{63}-1$ so independent implementations have an explicit accepted range, but intermediate arithmetic is not restricted to signed 64-bit values. Every interval is half open. Thus $[l, h) = \{l, l+1, \dots, h-1\}$, its cardinality is $h-l$, and $[l, l)$ is empty.

A canonical interval list is sorted by lower endpoint, has $0 \leq l < h \leq N$, and contains neither overlapping nor adjacent intervals. For consecutive intervals $[l_i, h_i)$ and $[l_{i+1}, h_{i+1})$, canonicity requires $h_i < l_{i+1}$. Requiring a gap, rather than merely nonoverlap, ensures that $[0, 2), [2, 4)$ is rejected in favor of the unique representation $[0, 4)$. The validator does not silently rewrite inputs before hashing.

B. Affine target map

The target universe is $\mathcal{U}_N = \{0, \dots, N-1\}$. A declaration contains $a, b \in \mathcal{U}_N$ with $\gcd(a, N) = 1$ and defines

$$\pi(i) = (ai + b) \bmod N. \quad (1)$$

There is an inverse $\pi^{-1}(x) = a^{-1}(x - b) \bmod N$. The checker computes a^{-1} only after validating coprimality. The word “permutation” in the artifact always refers to this algebraic bijection. It does not imply pseudorandomness, secrecy, or cryptographic strength.

C. Fragments and phases

A fragment has an identifier, a phase in $\{\text{committed}, \text{planned}\}$, bounds $0 \leq l < h \leq N$, stride $s \geq 1$, and residue $0 \leq r < s$. It selects

$$S(l, h, s, r) = \{i : l \leq i < h \wedge i \equiv r \pmod{s}\}. \quad (2)$$

Fragments in both phases contribute to target multiplicity. The phase is provenance useful to an operator: an overlap between committed and planned fragments can reveal an incorrect resume point. It has no special case in the safety equation.

The plan may have zero fragments. Each listed fragment must select at least one counter. Empty work is represented by omission rather than by degenerate bounds or an impossible congruence. Identifiers are unique nonempty strings and appear in strictly increasing lexical order. Ordering makes certificate rows and witnesses deterministic.

D. Authorization and multiplicity

The declaration’s canonical interval list denotes $\mathcal{A} \subseteq \mathcal{U}_N$. Every target outside $\mathcal{A}$ is forbidden by this model. Let

$$m_x = |\{f \in \mathcal{F} : x \in \pi(S_f)\}|. \quad (3)$$

A plan is safe exactly when

$$\forall x \in \mathcal{A} : m_x = 1, \quad \forall x \in \mathcal{U}_N \setminus \mathcal{A} : m_x = 0. \quad (4)$$

This is a property of the declared sets. It says nothing about whether a worker executes a set, executes it once, observes a stable remote state, or commits a side effect.

II. DEFECT IDENTITY: EXPANDED PROOF

Define

$$B = \sum_{f \in \mathcal{F}} |S_f|, \quad (5)$$

$$T = |\mathcal{A}|, \quad (6)$$

$$C = \sum_{f \in \mathcal{F}} |\pi(S_f) \cap \mathcal{A}|, \quad (7)$$

$$Q = \sum_{\{f,g\} \subseteq \mathcal{F}, f < g} |S_f \cap S_g|. \quad (8)$$

The order relation in Q selects each unordered pair once.

Lemma 1 (First moment). $B = \sum_{x \in \mathcal{U}_N} m_x$.

Proof. A fragment contains no repeated counter and π is injective, so it contributes one visit to each target in $\pi(S_f)$. Counting the incidence pairs (f, x) first by fragment gives B and first by target gives $\sum_x m_x$. $\square$

Lemma 2 (Second combinatorial moment). $Q = \sum_{x \in \mathcal{U}_N} \binom{m_x}{2}$.

Proof. Because π is injective, x belongs to both $\pi(S_f)$ and $\pi(S_g)$ exactly when $\pi^{-1}(x)$ belongs to $S_f \cap S_g$. For a fixed target visited by m_x fragments, exactly $\binom{m_x}{2}$ unordered fragment pairs intersect at its preimage. Counting triples $(\{f, g\}, x)$ by pair or by target proves the result. $\square$

Lemma 3 (Authorized first moment). $C = \sum_{x \in \mathcal{A}} m_x$.

Proof. Count incidence pairs (f, x) restricted to $x \in \mathcal{A}$ by fragment and by target. $\square$

Theorem 1 (Nonnegative defect).

$$B + T - 2C + 2Q = \sum_{x \in \mathcal{A}} (m_x - 1)^2 + \sum_{x \notin \mathcal{A}} m_x^2. \quad (9)$$

Proof. The first two lemmas yield

$$B + 2Q = \sum_x \left(m_x + 2 \binom{m_x}{2} \right) \quad (10)$$

$$= \sum_x (m_x + m_x(m_x - 1)) \quad (11)$$

$$= \sum_x m_x^2. \quad (12)$$

The authorized first moment and $T = \sum_{x \in \mathcal{A}} 1$ give

$$B + T - 2C + 2Q = \sum_x m_x^2 + \sum_{x \in \mathcal{A}} 1 - 2 \sum_{x \in \mathcal{A}} m_x \quad (13)$$

$$= \sum_{x \in \mathcal{A}} (m_x^2 - 2m_x + 1) + \sum_{x \notin \mathcal{A}} m_x^2. \quad (14)$$

Every summand is the square of an integer. $\square$

Proposition 1 (Decision equivalence). *The plan is safe if and only if $D = B + T - 2C + 2Q = 0$.*

Proof. If safe, every authorized square is $(1 - 1)^2 = 0$ and every unauthorized square is $0^2 = 0$. Conversely, a sum of nonnegative integers is zero only if every term is zero. Thus authorized multiplicities are one and unauthorized multiplicities are zero. $\square$

Two weaker checks illustrate why all terms are needed. Cardinality equality $B = T$ admits one missing and one duplicated authorized target. Equality $C = T$ can coexist with unauthorized visits if authorized omissions and duplicates balance the first moment. Pair information in Q converts the first moment into the second moment and prevents such cancellation.

III. COUNTING ONE FRAGMENT

A. Bounded congruence normalization

For $S(l, h, s, r)$, let

$$u = l + ((r - l) \bmod s). \quad (15)$$

This is the least integer $u \geq l$ congruent to r modulo s . If $u \geq h$, the set is empty. Otherwise its members are $u + sj$ for $0 \leq j < n$, where

$$n = 1 + \left\lfloor \frac{h - 1 - u}{s} \right\rfloor. \quad (16)$$

The implementation uses this formula for fragment cardinality and for the cardinality of a CRT intersection.

B. Signed floor sum

Define

$$F(n, m, a, b) = \sum_{j=0}^{n-1} \left\lfloor \frac{aj + b}{m} \right\rfloor, \quad n \geq 0, m > 0. \quad (17)$$

For signed a, b , Euclidean division gives $a = q_a m + r_a$ and $b = q_b m + r_b$ with $0 \leq r_a, r_b < m$. Therefore

$$F(n, m, a, b) = q_a \frac{n(n-1)}{2} + q_b n + F(n, m, r_a, r_b). \quad (18)$$

The remaining nonnegative floor sum is evaluated by repeatedly extracting quotients when $a \geq m$ or $b \geq m$ and applying the reciprocal reduction to the residual triangle. The release tests the signed wrapper against direct summation over a dense small range including negative coefficients.

C. Residue threshold formula

Let $z_j = \alpha j + \beta$ and $y_j = z_j \bmod N$. For $0 \leq t \leq N$,

$$I_j(t) = \left\lfloor \frac{z_j + N - t}{N} \right\rfloor - \left\lfloor \frac{z_j}{N} \right\rfloor \quad (19)$$

is one exactly when $y_j \geq t$. To see this, write $z_j = qN + y_j$ with $0 \leq y_j < N$; then $I_j(t) = \lfloor (y_j + N - t)/N \rfloor$. Consequently

$$|\{j : y_j < t\}| = n - \sum_j I_j(t) \quad (20)$$

$$= n - F(n, N, \alpha, \beta + N - t) + F(n, N, \alpha, \beta). \quad (21)$$

The endpoint cases return zero for $t = 0$ and n for $t = N$. The count in $[L, R)$ is the threshold count at R minus the threshold count at L . For a fragment, $\alpha = as$ and $\beta = au + b$.

The authorization count for a fragment is the sum over canonical mask intervals. Intervals are disjoint, so no inclusion–exclusion is required. The algorithm’s work is proportional to mask representation size, not mask cardinality.

IV. COUNTING FRAGMENT PAIRS

Two congruences

$$i \equiv r_1 \pmod{s_1}, \quad i \equiv r_2 \pmod{s_2} \quad (22)$$

are compatible if and only if $g = \gcd(s_1, s_2)$ divides $r_2 - r_1$. When compatible, divide by g and solve

$$(s_1/g)t \equiv (r_2 - r_1)/g \pmod{s_2/g}. \quad (23)$$

The coefficient is invertible modulo s_2/g , so a solution is

$$t = \frac{r_2 - r_1}{g} (s_1/g)^{-1} \pmod{s_2/g}. \quad (24)$$

Then $r = r_1 + s_1 t$ is reduced modulo $\text{lcm}(s_1, s_2)$ and describes every simultaneous solution. Intersecting the two counter bounds and applying bounded-progression normalization gives $|S_1 \cap S_2|$.

For full defect calculation this cardinality is sufficient. For a target prefix $[0, p)$, the intersection progression is mapped through the same affine permutation and counted with the threshold formula. This reuse is important: witness generation does not introduce an untested counting primitive.

The least common multiple can exceed the interchange bound on N even though the bounded intersection lies inside $[0, N)$. The Python reference uses arbitrary-precision integers. A fixed-width port must detect multiplication overflow before forming $(s_1/g)s_2$ or establish a semantically equivalent bounded algorithm.

V. PREFIX DEFECTS AND WITNESSES

For $0 \leq p \leq N$, define

$$D(p) = \sum_{\substack{x < p \\ x \in \mathcal{A}}} (m_x - 1)^2 + \sum_{\substack{x < p \\ x \notin \mathcal{A}}} m_x^2. \quad (25)$$

Then $D(0) = 0$, $D(N) = D$, and

$$D(p+1) - D(p) = \begin{cases} (m_p - 1)^2, & p \in \mathcal{A}, \\ m_p^2, & p \notin \mathcal{A}. \end{cases} \quad (26)$$

Thus $D(p)$ is monotone. When $D > 0$, binary search on $p \in [1, N]$ yields the smallest prefix with positive defect. The target $x = p - 1$ is the least target with an incorrect multiplicity.

The witness contains:

- schema identifier and canonical input digests;
- category, target x , and inverse source $\pi^{-1}(x)$;
- authorization Boolean, expected multiplicity, and actual multiplicity;
- lexically sorted covering fragment identifiers;
- $D(x)$, $D(x+1)$, and $D(N)$; and
- a digest of all preceding witness fields.

The checker reconstructs the entire canonical object. Listing a target that is defective but not minimal is rejected. A changed covering list, category, source counter, or prefix value is rejected even if another field remains plausible.

Witness generation is $O(\log N)$ full prefix analyses. A more elaborate implementation could cache invariant pair structure or descend an interval tree, but the reference favors a single transparent path. Witness production is invoked only after the one-pass full analysis has established $D > 0$.

VI. NORMATIVE JSON CONTRACTS

A. Declaration

The exact object keys are `schema`, `universe`, `permutation`, and `authorized`. The `schema` value is `pcs-declaration-v2`. The `permutation` object has exactly `kind`, `multiplier`, and `offset`, with `kind` `affine`. The `authorized` value is an array of two-element integer arrays. No nulls, numeric strings, floats, booleans, duplicate keys, unknown keys, or noncanonical interval lists are accepted.

```
{
  "schema": "pcs-declaration-v2",
  "universe": 64,
  "permutation": {
    "kind": "affine", "multiplier": 5, "offset": 7
  },
  "authorized": [[1,2], [3,4], ...]
}
```

B. Plan

The exact top-level keys are `schema` and `fragments`; `schema` is `pcs-plan-v2`. Each fragment has exactly `id`, `phase`, `lo`, `hi`, `stride`, and `residue`. The list is strictly ordered by identifier. A zero-fragment list is valid.

C. Certificate

A certificate has exactly $K(K-1)/2 + K + 2$ nonblank JSONL rows: one header, K fragment rows, one row for every ordered identifier pair $f < g$, and one summary. Each row is augmented with `seq`, `prev_sha256`, and `row_sha256`. The first previous hash is 64 zero digits. The row digest is SHA-256 of canonical JSON containing every row field except `row_sha256`; subsequent rows name that digest.

The checker does not merely validate a generic row schema. It generates the expected row at each position and compares exact canonical objects. This rejects an extra unknown field even when the mathematical values otherwise agree.

D. Canonical encoding and binding

Canonical JSON is UTF-8, with object keys sorted, separators `,` and `:` without surrounding whitespace, arrays in declared order, JSON literals in lowercase, and no ASCII escaping of valid non-ASCII strings. Inputs are validated before canonical encoding. The header contains lowercase hexadecimal SHA-256 digests of the complete canonical declaration and plan objects.

These digests protect association under the collision-resistance assumption. They do not identify an author, establish freshness, or prevent an attacker from replacing inputs and generating a new matching certificate. An authenticated outer envelope is required when those properties matter.

VII. CHECKER SOUNDNESS AND COMPLETENESS

Theorem 2 (Relative soundness). *Assume the checker implementation faithfully realizes the specified integer algorithms, the declaration and plan presented to it are the intended inputs, and SHA-256 collision resistance holds for input binding. If the checker accepts a certificate with summary `safe=true`, then the declared plan is permutation-complete for the declared authorization mask.*

Proof. Acceptance means strict input validation succeeded, header digests equal canonical input digests, every expected row was present in the unique position, and every row equaled the checker-recomputed row. Therefore the accepted summary's B, T, C, Q, D equal the mathematical quantities for those inputs. The checker sets `safe` exactly when $D = 0$. Decision equivalence then gives the required multiplicities. $\square$

Theorem 3 (Completeness). *For every valid declaration and plan, the reference producer’s certificate is accepted by the reference checker; its safe Boolean equals the semantic safety predicate.*

Proof. Producer and checker use the same deterministic canonical row construction on validated inputs. All integer counts are total for the accepted ranges, so generated rows compare equal. Decision equivalence establishes the Boolean result. $\square$

These theorems are relative to implementation fidelity; the source code is not extracted from a proof assistant. Differential testing, strict rejection tests, and clean builds increase confidence but are not a machine-checked refinement proof.

VIII. INDEPENDENT ORACLE AND TEST DESIGN

A. Explicit oracle

For $N \leq 10^6$, the oracle allocates an array $m[0 \dots N-1]$ initialized to zero. It independently enumerates each fragment by testing or stepping through its bounded congruence, computes $(ai + b) \bmod N$, and increments the corresponding target. It then directly evaluates the target-wise squares. It does not call the optimized floor-sum, mapped-interval, CRT, aggregate-analysis, certificate, or witness code.

Shared strict parsing is not used inside the randomized semantic test: objects are constructed in memory and validated before both paths. This arrangement catches arithmetic disagreement but could still share a mistaken high-level interpretation. The normative semantics and retained small examples support human inspection of that layer.

B. Deterministic test families

TABLE I: Principal test families.

Family	Obligation
Signed floor sum	Exhaustive direct summation for small n, m and negative/positive a, b .
Residue threshold	5,000 seeded random cases compared with explicit modular residues.
Mapped target interval	5,000 seeded fragments, affine maps, and half-open target intervals compared with enumeration.
Generalized CRT	3,000 seeded pairs compared with explicit bounded set intersection.
Complete plan	1,000 seeded declarations/plans compared target by target with the independent oracle.
Witnesses	Missing, duplicate, forbidden, safe-no-witness, inverse source, prefix monotonicity, and tampering.
Strict parser	Duplicate keys, wrong schema, unknown fields, booleans as integers, invalid intervals, phases, identifiers, and empty fragments.
Certificate structure	Round trip, input mismatch, row deletion/duplication/reordering, field edits, unknown fields, chain edits, truncation, and blank lines.
Tamper matrix	200 seeded mutations across Boolean, integer, and string certificate fields; every mutation must be rejected.
CLI	Successful creation/check, malformed input status, and valid-but-unsafe <code>-require-safe</code> status.

Random generators use fixed seeds recorded in source. A failed case therefore reproduces without a property-testing framework. Tests use only the Python standard library and run through `unittest`.

IX. BENCHMARK CONSTRUCTION

The scaling cases are semantic fixtures, not random workloads. For a power-of-two universe, the generator chooses an odd affine multiplier, splits the source range into widely separated blocks, and creates one stride-one fragment per block. Blocks are disjoint. It maps the selected source points through the affine permutation and canonicalizes those images into the authorization interval list. Thus every selected point is authorized exactly once and no other point is selected; expected $D = 0$ follows directly from construction.

This method produces masks with many isolated intervals while keeping point counts manageable. The focal case has universe 2^{32} , 125 fragments, and 1,024 authorized points. Certificate construction still evaluates every fragment-mask combination and every fragment pair, so the case exercises the structural terms while avoiding a universe-sized ground-truth file.

For the enumeration baseline, structure is fixed at 32 fragments and 256 authorized points while N ranges from 2^{12} to 2^{20} . The explicit oracle’s multiplicity array grows with N ; optimized arithmetic does not enumerate it. Both paths must return exact zero before a timing is retained.

Each timed operation runs repeatedly in one process and reports all repeats plus median, minimum, maximum, and quartiles. The script uses wall-clock nanoseconds. Python’s `tracemalloc` measures peak traced Python allocations for one verification call. It does not measure total resident set size, mapped libraries, kernel buffers, or filesystem cache; the paper labels it accordingly.

X. REPRODUCTION PROTOCOL

From the artifact root, the normative command is:

```
python software/scripts/reproduce.py
```

The script checks the Python version, runs the unit suite, regenerates the four small examples, validates every retained certificate and witness, reruns semantic benchmarks, recompiles the paper and supplement when TeX tools are available, scans logs and extracted text, and writes a machine-readable reproduction report. It returns nonzero on an exact semantic or build failure.

For a fast manual path:

```
PYTHONPATH=software python -m unittest discover \
-s software/tests -v
PYTHONPATH=software python software/pcs.py check \
examples/declaration.json examples/safe-plan.json \
examples/safe-plan-certificate.jsonl --require-safe
PYTHONPATH=software python software/pcs.py check-witness \
examples/declaration.json examples/missing-plan.json \
examples/missing-plan-witness.json
```

The top-level manifest gives SHA-256 values for release files. ZIP integrity is checked separately with the archive’s CRC records and path-safety rules. A clean-room test extracts the public artifact into a new temporary directory and invokes the same reproduction entry point without referring to the source work tree.

XI. CLAIM-EVIDENCE BOUNDARIES

TABLE II: Claim, evidence, and non-claim.

Claim	Evidence	Explicit non-claim
$D = 0$ iff masked exact coverage	Algebraic theorem and small explicit cases	No assertion about arbitrary execution traces
Mapped interval counts are exact	Threshold lemma, signed floor sum, 5,000 differential cases	No support for arbitrary PRPs

Claim	Evidence	Explicit non-claim
Pair counts are exact	Generalized CRT derivation and 3,000 differential cases	No data-dependent fragment predicates
Certificate is input-bound	Canonical SHA-256 fields and mismatch tests	No signer identity or freshness
Transcript tampering is rejected	Full recomputation plus fixed and randomized mutations	No protection if checker/runtime is compromised
Witness is earliest	Prefix-square proof and canonical recomputation	Not the earliest wall-clock execution failure
Large universe is feasible for retained structure	Raw benchmark inputs/repeats and clean rerun	No universal latency guarantee
Bibliographic records are real	Frozen Crossref responses and exact DOI matching	No claim that metadata verifies scientific conclusions

XII. KNOWN LIMITATIONS

The fragment language cannot directly encode data-dependent skips or a union of many residues under one identifier. The interval mask can represent any finite subset but may require one interval per isolated point. Pair rows scale quadratically in fragment count. Prefix witnesses multiply analysis work by $O(\log N)$. The Python checker is not constant time, formally verified, or hardened as a network service. Hash binding is unsigned. The benchmark is synthetic and single-host. The reference audit depends on a metadata service and verifies identity rather than interpretation. These limits are recorded in both human- and machine-readable release documents.

XIII. REVIEWER CHECKLIST

A reviewer can complete the following independent checks without trusting the paper’s prose:

- 1) confirm the main PDF has exactly 12 pages and no unresolved citations;
- 2) run the unit suite and inspect the reported test count and OK status;
- 3) check the safe example, then edit one certificate field and confirm rejection;
- 4) recompute one witness of each category and compare target/minimality fields;
- 5) inspect the explicit oracle to confirm it does not call optimized counters;
- 6) recompute D manually for the 16-position fixed fixtures;
- 7) inspect `results.json` and regenerate paper table rows;
- 8) verify at least 40 unique DOI records and their frozen raw responses;
- 9) extract the public ZIP into an empty directory and run reproduction; and
- 10) compare release files against the external SHA-256 manifest.

XIV. CONCLUSION

The supplement makes the contract auditable at four levels: set semantics, integer derivations, byte-level formats, and executable evidence. The guarantee remains precise: for the declared affine map, interval authorization mask, and bounded-congruence fragments, an accepted zero-defect certificate establishes exact masked plan coverage. All broader operational and cryptographic properties require separate mechanisms.